

Desarrollo de Software

2024 - 2025



Ejercicio Guiado 3

Alejandro Pérez Montes 100499978

Alejandro de Santos Lobo 100499148

Grupo 801 Subgrupo 03

ÍNDICE GENERAL

1. POTENCIAL DE LOS LLMS PARA REFACTORIZAR	1
2. ESTIMACIÓN Y CUANTIFICACIÓN DE LOS BENEFICIOS DE LA RE- FACTORIZACIÓN	5
3. HACIA SISTEMAS DE SOFTWARE EN LA NUBE SOSTENIBLES A TRA- VÉS DE LA REFACTORIZACIÓN DE 'CODE SMELLS' CONSCIENTE DE LA ENERGÍA	7
4. EVALUACIÓN EMPÍRICA DEL IMPACTO DE LA REFACTORIZACIÓN DE CÓDIGO ORIENTADO A OBJETOS EN LOS ATRIBUTOS DE CALIDAD .	9
4.0.1. Escenarios de Refactorización (RQ1)	9
4.0.2. Atributos de Calidad (RQ2)	9
4.0.3. Impacto de la Refactorización en la Calidad (RQ5).	10

1. POTENCIAL DE LOS LLM'S PARA REFACTORIZAR

El presente estudio empírico [1], llevado a cabo por un equipo de investigadores de diversas instituciones chinas y publicado en noviembre del 2024, exploró la capacidad de los Modelos de Lenguaje Grandes (LLMs), específicamente ChatGPT (GPT-4) y Gemini (Gemini-1.0 Pro), en el ámbito de la refactorización automatizada de software.

La investigación se basó en un conjunto de datos de alta calidad que comprendía 180 refactorizaciones atómicas identificadas en 20 proyectos de software. Las refactorizaciones consideradas en este estudio se limitaron a ocho categorías, a saber: extraer métodos, extraer variables, métodos en misma línea, variables en misma línea, renombrar atributos, renombrar métodos, renombrar parámetros y renombrar variables.

Cabe señalar que este estudio no abarcó técnicas más complejas como las relacionadas con Clases Divergentes, que podrían requerir la reubicación de métodos a superclases o la aplicación de patrones de herencia y polimorfismo.

Dentro del marco de este estudio, se llevaron a cabo dos experimentos principales para evaluar el rendimiento de los LLMs en la identificación de oportunidades de refactorización y la proposición de soluciones de código refactorizado.

Prompt 1 (P1):
As a developer, imagine your team leader requests you to review a piece of code to identify potential refactoring opportunities. The original code snippet is as follows: ''' <code>original code</code> ''' If there are any refactoring opportunities, please do it and generate the refactored code. Otherwise, simply state that no refactoring is necessary.
Prompt 2 (P2):
Rename method refactorings are frequently employed to modify low-quality identifiers to improve readability. As a developer, imagine your team leader requests you to review a piece of code to identify potential rename method refactoring opportunities. The original code snippet is as follows: ''' <code>original code</code> ''' If there are any refactoring opportunities, please do it and generate the refactored code. Otherwise, simply state that no refactoring is necessary.

Fig. 1.1. Prompts empleados en los experimentos

En el primer experimento, se utilizó un enfoque directo mediante el Prompt1 (P1). Este prompt asignaba un rol genérico al LLM (el de desarrollador) y le proporcionaba el código fuente como entrada, solicitándole una tarea general como "encuentra oportunidades de refactorización y refactoriza". Los resultados obtenidos con este enfoque inicial fueron modestos, con tasas de éxito bajas en la identificación de refactorizaciones en ambos modelos.

Table 1. LLM-based Identification of Refactoring Opportunities

Refactoring Type	GPT		Gemini	
	#Succeeded	#Missed	#Succeeded	#Missed
Extract Class	0 (0%)	20	0 (0%)	20
Extract Method	9 (45%)	11	1 (5%)	19
Extract Variable	5 (25%)	15	0 (0%)	20
Inline Method	3 (15%)	17	3 (15%)	17
Inline Variable	3 (15%)	17	1 (5%)	19
Rename Attribute	1 (5%)	19	0 (0%)	20
Rename Method	2 (10%)	18	0 (0%)	20
Rename Parameter	3 (15%)	17	0 (0%)	20
Rename Variable	2 (10%)	18	2 (10%)	18
Total	28 (15.6%)	152	7 (3.9%)	173

Fig. 1.2. Resultados de usar P1

El segundo experimento, empleando el Prompt2 (P2), demostró una mejora significativa en el rendimiento de los LLMs. El prompt2, a diferencia del prompt1, especificaba explícitamente el tipo de refactorización que el LLM debía buscar en el código fuente proporcionado.

Table 2. LLM-based Identification of Refactoring Opportunities (Specifying Expected Refactoring Types)

Refactoring Type	GPT		Gemini	
	#Succeeded	#Missed	#Succeeded	#Missed
Extract Class	9 (45%)	11	2 (10%)	18
Extract Method	12 (60%)	8	5 (25%)	15
Extract Variable	7 (35%)	13	1 (5%)	19
Inline Method	7 (35%)	13	6 (30%)	14
Inline Variable	5 (25%)	15	3 (15%)	17
Rename Attribute	14 (70%)	6	8 (40%)	12
Rename Method	12 (60%)	8	6 (30%)	14
Rename Parameter	14 (70%)	6	4 (20%)	16
Rename Variable	14 (70%)	6	3 (15%)	17
Total	94 (52.2%)	86	38 (21.1%)	142

Fig. 1.3. Resultados de usar P2

Posteriormente, para evaluar el impacto de estrategias de prompting más elaboradas, se desarrollaron versiones mejoradas del Prompt2 que se midieron únicamente con GPT. La primera versión mejorada ("P2 + Refactoring Subcategory") incluía la especificación de subcategorías de refactorización junto con una explicación detallada de la técnica especificada con un ejemplo. La segunda versión mejorada ("P2 + Refactoring Subcategory + Narrowing Search Space") incluía todo lo anterior y además acotaba el espacio de búsqueda.

Table 3. LLM-based Identification of Refactoring Opportunities (Specifying Expected Refactoring Subcategories (\$) And Narrowing the Search Space (†))

Refactoring Type	P2		P2 ^{\$}		P2 ^{\$†}	
	#Succeeded	#Missed	#Succeeded	#Missed	#Succeeded	#Missed
Extract Class	9 (45%)	11	10 (50%)	10	10 (50%)	10
Extract Method	12 (60%)	8	14 (70%)	6	19 (95%)	1
Extract Variable	7 (35%)	13	13 (65%)	7	20 (100%)	0
Inline Method	7 (35%)	13	12 (60%)	8	20 (100%)	0
Inline Variable	5 (25%)	15	7 (35%)	13	14 (70%)	6
Rename Attribute	14 (70%)	6	16 (80%)	4	17 (85%)	3
Rename Method	12 (60%)	8	15 (75%)	5	19 (95%)	1
Rename Parameter	14 (70%)	6	17 (85%)	3	19 (95%)	1
Rename Variable	14 (70%)	6	16 (80%)	4	18 (90%)	2
Total	94 (52.2%)	86	120 (66.7%)	60	156 (86.7%)	24

Fig. 1.4. Resultados de usar P2 y sus versiones mejoradas

La justificación para acotar el espacio de búsqueda se encuentra en la observación, de que el rendimiento de los LLMs en la identificación de refactorizaciones tiende a disminuir a medida que aumenta el tamaño del código fuente de entrada.

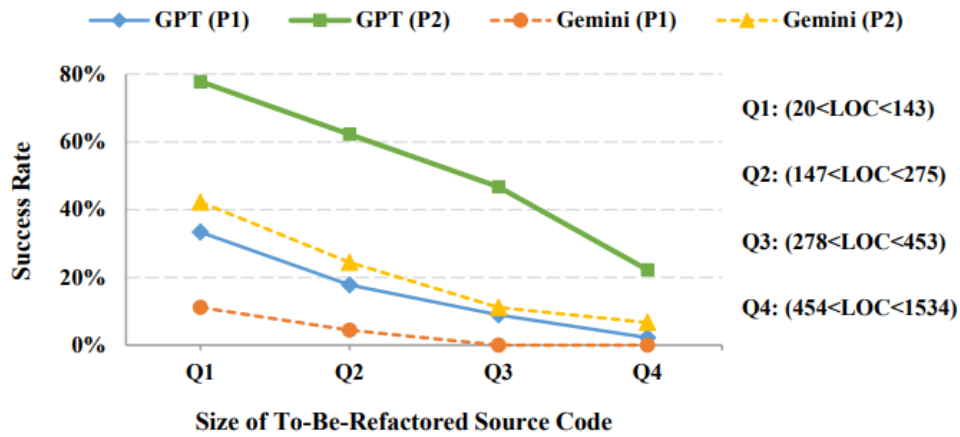


Fig. 1.5. Tasa de Éxito en función de diferentes tamaños del código fuente

Ante el riesgo de que los LLMs generen refactorizaciones inseguras que alteren la funcionalidad del código o introduzcan errores de sintaxis, los autores proponen una táctica de "detectar y reaplicar" denominada Refactoring Mirror cuyo objetivo es mitigar estos riesgos aprovechando la inteligencia de los LLMs para identificar refactorizaciones beneficiosas, pero garantizando la seguridad mediante el uso de herramientas de refactorización probadas como IntelliJ IDEA.

2. ESTIMACIÓN Y CUANTIFICACIÓN DE LOS BENEFICIOS DE LA REFACTORIZACIÓN

El objetivo principal de esta investigación [2] fue estimar cuantitativamente los beneficios de modularizar un componente de software. Específicamente, los investigadores buscaron responder a la pregunta: "¿Cómo podemos estimar, de manera cuantitativa, los beneficios de modularizar un componente?"

La investigación se llevó a cabo en una gran empresa de software del sector de las telecomunicaciones. Los investigadores utilizaron un enfoque de métodos mixtos, recopilando datos cualitativos y cuantitativos a través de datos de la compañía, encuestas y entrevistas. El estudio se centró en comparar un componente no refactorizado (NRC) con un nuevo componente (NC) que utilizaba funcionalidad extraída y modularizada en un framework separado (FW). A partir de comparar datos de esfuerzo medido en Horas Hombre entre ambos componentes, modelaron una serie de ecuaciones que pretenden facilitar la toma de decisiones a la hora de refactorizar un módulo.

El estudio concluyó que la refactorización para mejorar la modularidad puede proporcionar beneficios significativos en términos de esfuerzo de desarrollo y mantenimiento, aunque estos beneficios pueden tardar en materializarse a largo plazo.

Si bien, el mayor valor de esta investigación es que proporciona a los profesionales un método y una fórmula para cuantificar el valor de negocio de la modularización, ayudando a los interesados a tomar decisiones informadas sobre si invertir o no en refactorización. Ofrece una forma de ir más allá de simplemente ver la refactorización como un coste y, en cambio, verla como una inversión potencial con retornos cuantificables. El método se puede adaptar a diferentes contextos y paradigmas de programación. También, cabe recordar que la fórmula planteada es una aproximación y su utilidad reside en el enfoque de toma de decisiones que plantea y no tanto su exactitud como ecuación.

Equation 2. Detailed formula considering the possible double maintenance

$$M(x) = \begin{cases} \frac{r-1}{r} E_2 + \frac{1}{r} E_5 & \text{if } \frac{x}{m} < 1 \\ \frac{r-2}{r} E_2 + \frac{2}{r} E_5 & \text{if } \frac{x}{m} < 2 \\ \vdots & \\ \frac{r-(r-1)}{r} E_2 + \frac{r-1}{r} E_5 & \text{if } \frac{x}{m} < r-1 \\ E_5 & \text{otherwise} \end{cases}$$

$$E = (E_1 - E_4) * t + \left(E_2 * t - \sum_{x=1}^t (M(x)) \right) - E_3$$

In summary, the important variables were the following:

- E: Effort Saved or Lost After t Months
- E1: Effort of Development Before Refactoring in Months
- E2: Effort of Maintenance Before Refactoring in Months
- E3: Total Effort of Doing the Refactoring
- E4: Effort of Development After Refactoring in Months
- E5: Effort of Maintenance After Refactoring in Months
- t: Time After the Refactoring in Months
- m: Months Between Releases
- r: Number of Releases Being Maintained at a Single Point in Time

Fig. 2.1. Formula para calcular la ganancia de esfuerzo

Esta formula devuelve el esfuerzo "ganado.º mejor dicho ahorrado, medido en Horas Hombre, tras computar las diferencias entre el tiempo de desarrollo $E_1 - E_4$ y de mantenimiento $E_2 \cdot t - \sum_{x=1}^t M(x)$ entre un módulo no refactorizado y otro refactorizado restando el tiempo invertido en refactorizar E_3 .

El término $\sum_{x=1}^t M(x)$ simboliza el esfuerzo de mantenimiento asociado a un módulo cada mes después de su lanzamiento. Dicho término presupone que se mantiene una versión antigua sin refactorizar y una nueva versión refactorizada.

Nótese, el ahorro de tiempo en desarrollo y mantenimiento están multiplicados por t reflejando que cuanto mayor es el ciclo de vida de dicho módulo de software, mayor es el ahorro en términos de esfuerzo generado. Dicho ahorro en la práctica se traduce en que los desarrolladores liberan horas de trabajo que se pueden destinar a otros proyectos alternativos.

3. HACIA SISTEMAS DE SOFTWARE EN LA NUBE SOSTENIBLES A TRAVÉS DE LA REFACTORIZACIÓN DE 'CODE SMELLS' CONSCIENTE DE LA ENERGÍA

Resumen

Este estudio [3] investiga el impacto de 16 tipos de *code smells* en el consumo energético de sistemas cloud, analizando 31 aplicaciones reales (Java/Python) mediante herramientas como *JDeodorant* y *PyCharm*. Hemos considerado relevante este artículo ya que tiene resultados interesantes, como que no siempre refactorizar reduce el consumo, por lo que debe estudiarse los beneficios de esta refactorización y concluir si conviene o no:

- Refactorizaciones como eliminar *Dead Code* reducen hasta un 13.1 % el consumo energético
- Técnicas como dividir *God Class* aumentan un 16 % el uso de CPU por sobrecarga en comunicación entre clases
- El *batch refactoring* de smells compatibles logra ahorros acumulativos de hasta 128.22 julios por workload

Relevancia

- **Innovación:** Primer estudio que cuantifica el impacto ambiental del refactoring en entornos cloud
- **Aplicabilidad:** Proporciona directrices para priorizar refactorizaciones según objetivos energéticos
- **Métricas:** Combina indicadores técnicos (CPU/memoria) con huella de carbono (gCO₂e)

Metodología

Code Smell	CPU	Memoria
Dead Code	↓ 12 %	↓ 8 %
God Class	↑ 16 %	↑ 9 %
Cyclic Dependency	↓ 8 %	↓ 5 %

TABLA 3.1. IMPACTO DE REFACTORIZACIONES
SELECCIONADAS

Conclusiones

Los autores demuestran que:

1. No todas las refactorizaciones mejoran la eficiencia energética
2. La selección inteligente de smells a refactorizar puede reducir hasta 5.1 % las emisiones de CO₂
3. Herramientas automatizadas deben incorporar métricas de sostenibilidad

4. EVALUACIÓN EMPÍRICA DEL IMPACTO DE LA REFACTORIZACIÓN DE CÓDIGO ORIENTADO A OBJETOS EN LOS ATRIBUTOS DE CALIDAD

Objetivo del Estudio

El artículo presenta una revisión sistemática de la literatura (SLR) que analiza el impacto de la refactorización de código orientado a objetos en atributos de calidad internos y externos del software [4]. El estudio agrega, resume y discute los resultados de 76 estudios primarios (PSs) relevantes publicados antes de finales de 2015.

Metodología

- **Selección de Estudios:** Se identificaron 2,259 artículos potencialmente relevantes, de los cuales 76 cumplieron con los criterios de inclusión.
- **Clasificación:** Los estudios se analizaron en función de:
 - Escenarios de refactorización (RQ1).
 - Atributos y medidas de calidad (RQ2).
 - Enfoques y técnicas estadísticas (RQ3).
 - Conjuntos de datos utilizados (RQ4).
 - Resultados del impacto de la refactorización en la calidad (RQ5).
- **Evaluación de Calidad:** Se aplicó un enfoque de conteo de votos para determinar la consistencia de los resultados reportados.

Hallazgos Clave

4.0.1. Escenarios de Refactorización (RQ1)

- Los escenarios más estudiados fueron *Move Method*, *Extract Class* y *Extract Method*.
- Solo el 50 % de los escenarios propuestos por Fowler fueron investigados.

4.0.2. Atributos de Calidad (RQ2)

- Los atributos internos más analizados fueron acoplamiento (*coupling*), cohesión (*cohesion*) y complejidad (*complexity*).

- Pocos estudios abordaron atributos externos medidos directamente, como mantenibilidad y confiabilidad.

4.0.3. Impacto de la Refactorización en la Calidad (RQ5)

- **Atributos Internos:**

- *Extract Class* mejora la cohesión pero puede aumentar el acoplamiento.
- *Move Method* mejora la cohesión y reduce el acoplamiento.
- *Extract Method* puede aumentar la complejidad y el tamaño del código.

- **Atributos Externos:**

- Los resultados fueron inconsistentes debido a la diversidad de métricas y enfoques utilizados.

- **Conclusión General:** La refactorización no siempre mejora todos los aspectos de la calidad del software; su impacto varía según el escenario y el atributo evaluado. Hemos elegido este artículo ya que investiga de forma muy genérica los impactos de la refactorización, y podemos ver como diversos métodos como Extract Class o Extract Method pueden tener impactos positivos y también negativos, por lo que es importante ser conocedor de ellos para poder diseñar software y refactorización de forma correcta.

BIBLIOGRAFÍA

- [1] B. Liu et al., «An Empirical Study on the Potential of LLMs in Automated Software Refactoring,» *arXiv preprint arXiv:2411.04444*, 2024.
- [2] A. Martini, E. Sikander y N. Medlani, «Estimating and Quantifying the Benefits of Refactoring to Improve a Component Modularity: A Case Study,» en *2016 42th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2016, pp. 92-99. doi: [10.1109/SEAA.2016.48](https://doi.org/10.1109/SEAA.2016.48).
- [3] A. Imran, T. Kosar, J. Zola y M. F. Bulut, *Towards Sustainable Cloud Software Systems through Energy-Aware Code Smell Refactoring*, 2024. doi: [10.1109/CLOUD62652.2024.00034](https://doi.org/10.1109/CLOUD62652.2024.00034). [En línea]. Disponible en: <https://ieeexplore.ieee.org/document/10643924>.
- [4] J. A. Dallal y A. Abdin, «Empirical Evaluation of the Impact of Object-Oriented Code Refactoring on Quality Attributes: A Systematic Literature Review,» *IEEE Transactions on Software Engineering*, vol. 44, n.º 1, pp. 44-68, 2018.